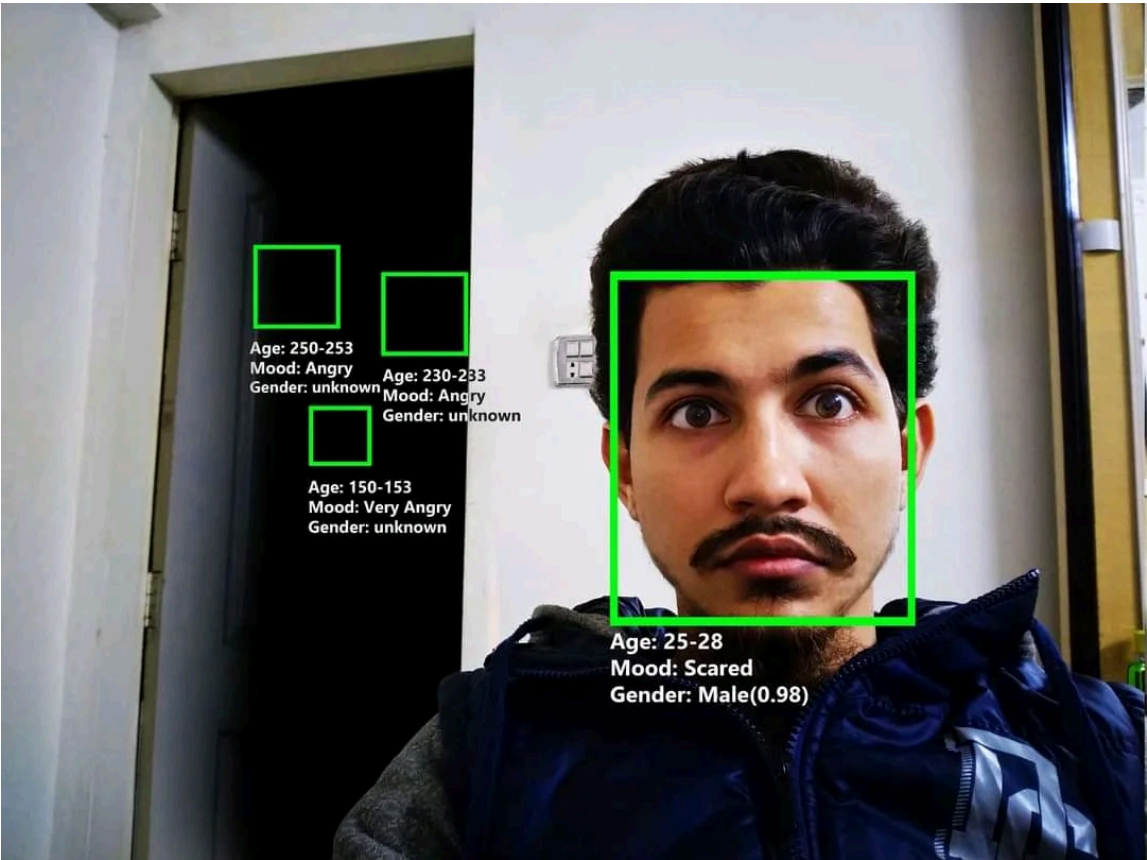


Supervised Learning

Clasificación Predictiva en Salud y Reconocimiento de Dígitos



En este proyecto se desarrollan dos aplicaciones de Aprendizaje Supervisado:

1. Un modelo de clasificación para estimar la probabilidad de que un paciente presente riesgo de ataque al corazón.
2. Un modelo de reconocimiento de dígitos manuscritos utilizando el dataset MNIST.

El desarrollo contempla el flujo completo de trabajo: análisis exploratorio, preparación de datos, entrenamiento de modelos, evaluación y generación de predicciones sobre nuevas observaciones.

Cada etapa será documentada detalladamente dentro del cuaderno para asegurar trazabilidad y claridad metodológica.

Exploratory Data Analysis

Trabajaremos con el conjunto de datos **Heart Attack**, cuyo objetivo es identificar qué variables clínicas influyen en la probabilidad de que un paciente sufra un evento cardiovascular.

Antes de construir un modelo predictivo, es fundamental comprender la estructura del dataset, el comportamiento de las variables y la relación entre ellas.

Variables del dataset

- **age:** Edad del paciente
- **sex:** Sexo del paciente
- **cp:** Tipo de dolor torácico (0–3)
- **trtbps:** Presión arterial en reposo (mm Hg)
- **chol:** Nivel de colesterol (mg/dl)
- **fbs:** Glucosa en ayunas > 120 mg/dl (1 = Sí, 0 = No)
- **restecg:** Resultado electrocardiográfico en reposo

- **thalachh**: Frecuencia cardíaca máxima alcanzada
- **oldpeak**: Depresión del segmento ST
- **slp**: Pendiente del segmento ST
- **caa**: Número de vasos principales afectados
- **thall**: Resultado de prueba de esfuerzo
- **exng**: Angina inducida por ejercicio (1 = Sí, 0 = No)
- **output**: Variable objetivo (1 = Riesgo, 0 = No riesgo)

El siguiente paso consiste en cargar el dataset y analizar su estructura.

```
In [28]: # Archivo Heart Attack.csv - Factores asociados al riesgo cardiovascular

import pandas as pd

# Ruta absoluta del proyecto
ruta_heart = r"D:\Documentos\Ebac\Finales\Tarea M25-CD -RobertScience\data26part2\H

# Carga del dataset
df = pd.read_csv(ruta_heart)

# Visualización inicial
print("Dimensiones del dataset:", df.shape)
df.head()
```

Dimensiones del dataset: (303, 14)

Out[28]:

	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	out
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	

```
In [29]: # =====
# Inspección estructural del dataset
# =====

print("\nInformación general:")
df.info()

print("\nResumen estadístico:")
df.describe()
```

Información general:
<class 'pandas.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
Column Non-Null Count Dtype

0 age 303 non-null int64
1 sex 303 non-null int64
2 cp 303 non-null int64
3 trtbps 303 non-null int64
4 chol 303 non-null int64
5 fbs 303 non-null int64
6 restecg 303 non-null int64
7 thalachh 303 non-null int64
8 exng 303 non-null int64
9 oldpeak 303 non-null float64
10 slp 303 non-null int64
11 caa 303 non-null int64
12 thall 303 non-null int64
13 output 303 non-null int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB

Resumen estadístico:

Out[29]:

	age	sex	cp	trtbps	chol	fbs	restecg
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000
mean	54.366337	0.683168	0.966997	131.623762	246.264026	0.148515	0.528053
std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.525860
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000
25%	47.500000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000
50%	55.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000
75%	61.000000	1.000000	2.000000	140.000000	274.500000	0.000000	1.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000

In [30]:

```
# =====  
# Verificación de valores nulos  
# =====  
  
valores_nulos = df.isnull().sum()  
  
print("Valores nulos por variable:\n")  
print(valores_nulos)  
  
print("\nTotal de valores nulos en el dataset:", valores_nulos.sum())
```

Valores nulos por variable:

age 0
sex 0
cp 0
trtbps 0
chol 0
fbs 0
restecg 0
thalachh 0
exng 0
oldpeak 0
slp 0
caa 0
thall 0
output 0
dtype: int64

Total de valores nulos en el dataset: 0

```
In [31]: # =====
# Distribución de la variable objetivo
# =====

print("Distribución absoluta del target:\n")
print(df["output"].value_counts())

print("\nDistribución porcentual:\n")
print(df["output"].value_counts(normalize=True) * 100)
```

Distribución absoluta del target:

```
output
1    165
0    138
Name: count, dtype: int64
```

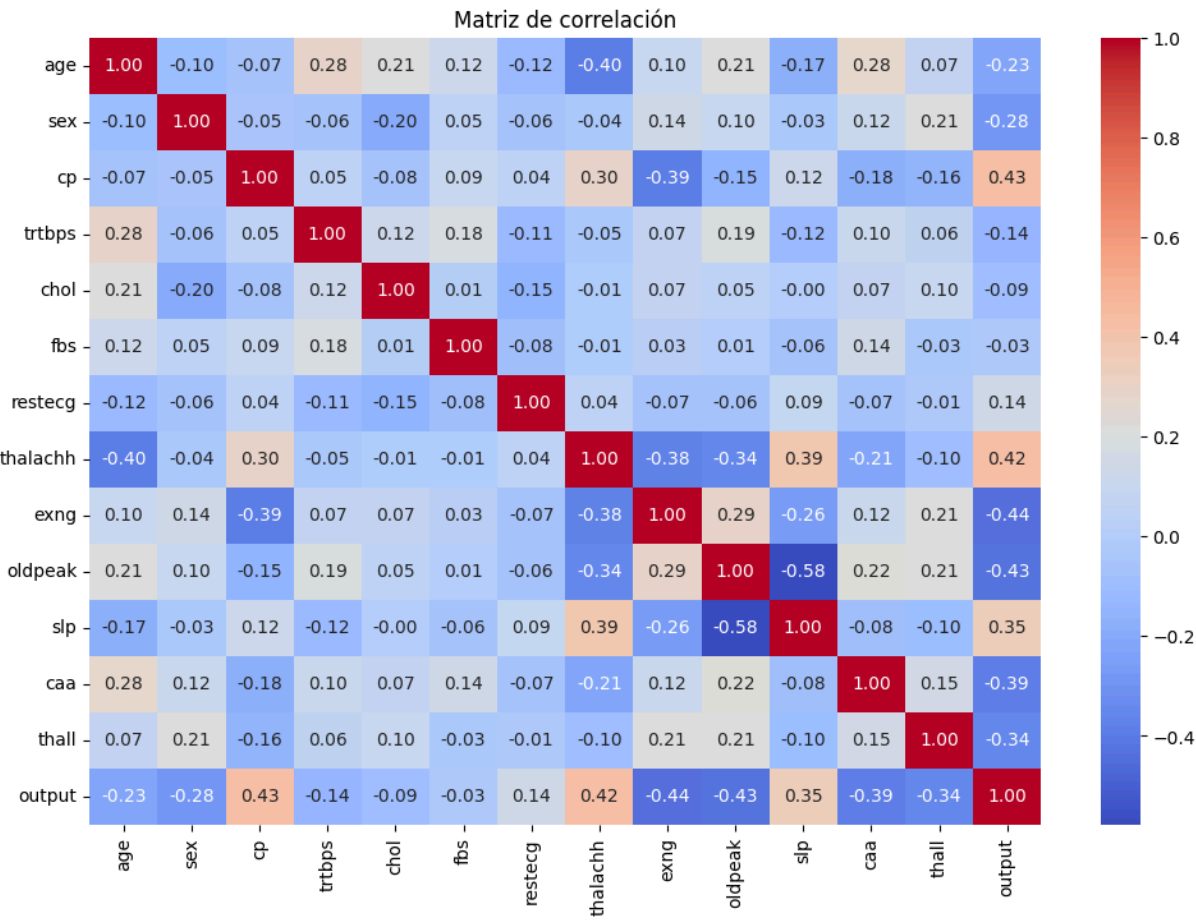
Distribución porcentual:

```
output
1    54.455446
0    45.544554
Name: proportion, dtype: float64
```

```
In [32]: # =====
# Matriz de correlación
# =====

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Matriz de correlación")
plt.show()
```



```
In [33]: # =====
# Correlación específica con la variable objetivo
# =====

correlacion_output = df.corr()["output"].sort_values(ascending=False)

print("Correlación de variables con el riesgo cardiovascular:\n")
print(correlacion_output)
```

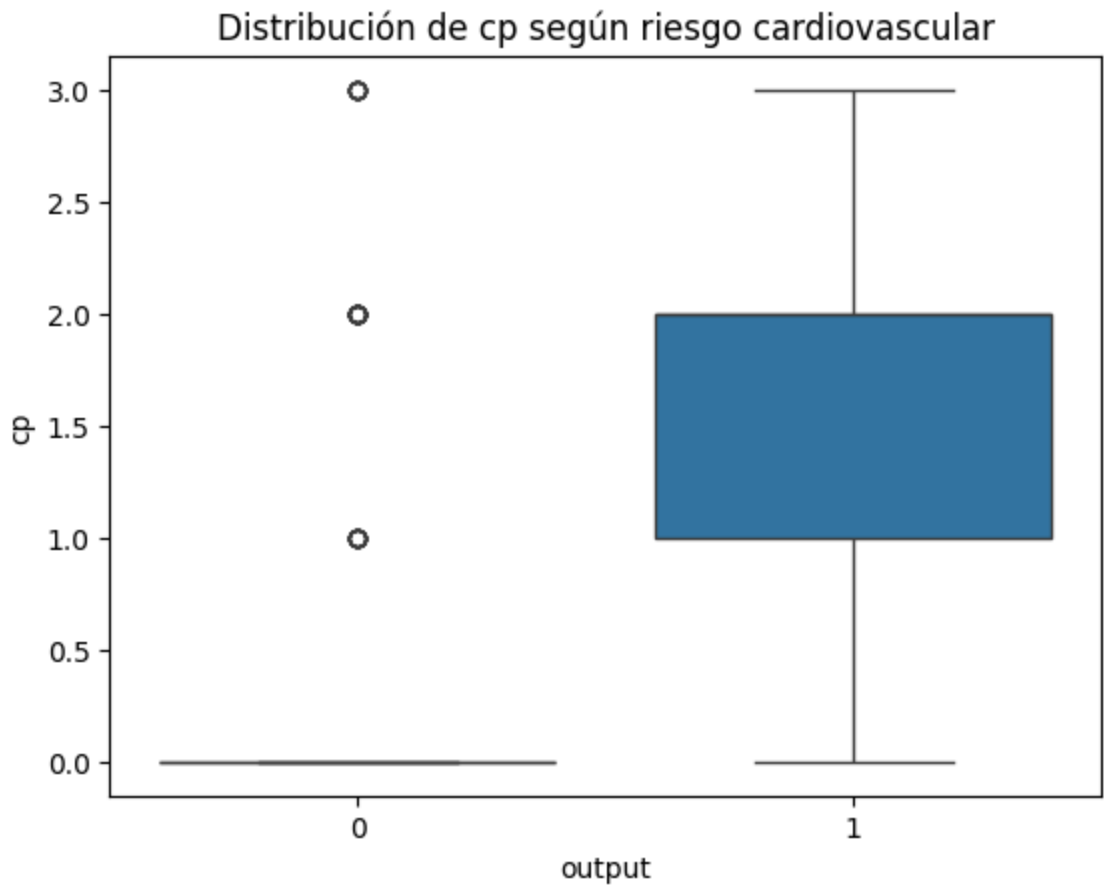
Correlación de variables con el riesgo cardiovascular:

```
output      1.000000
cp          0.433798
thalachh    0.421741
slp         0.345877
restecg     0.137230
fbs        -0.028046
chol       -0.085239
trtbps     -0.144931
age        -0.225439
sex        -0.280937
thall      -0.344029
caa        -0.391724
oldpeak    -0.430696
exng       -0.436757
Name: output, dtype: float64
```

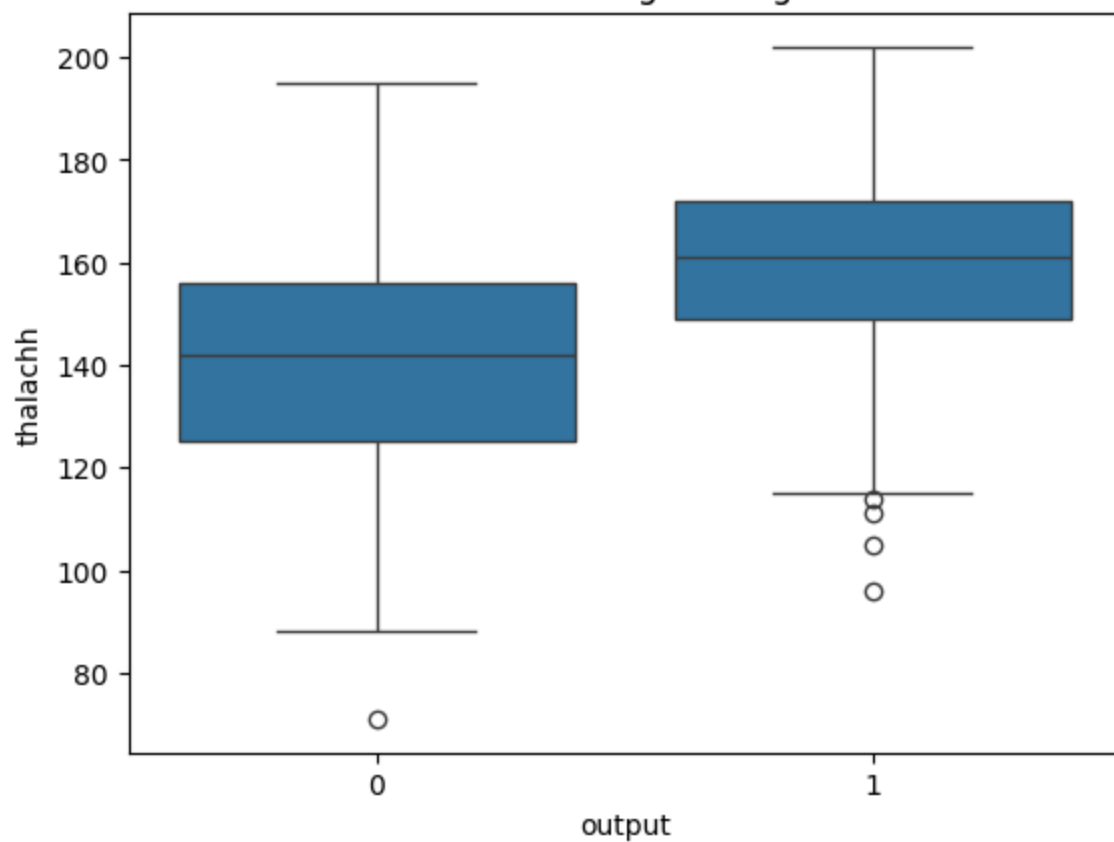
```
In [34]: # =====
# Visualización de variables relevantes
# =====

variables_clave = correlacion_output.index[1:6]

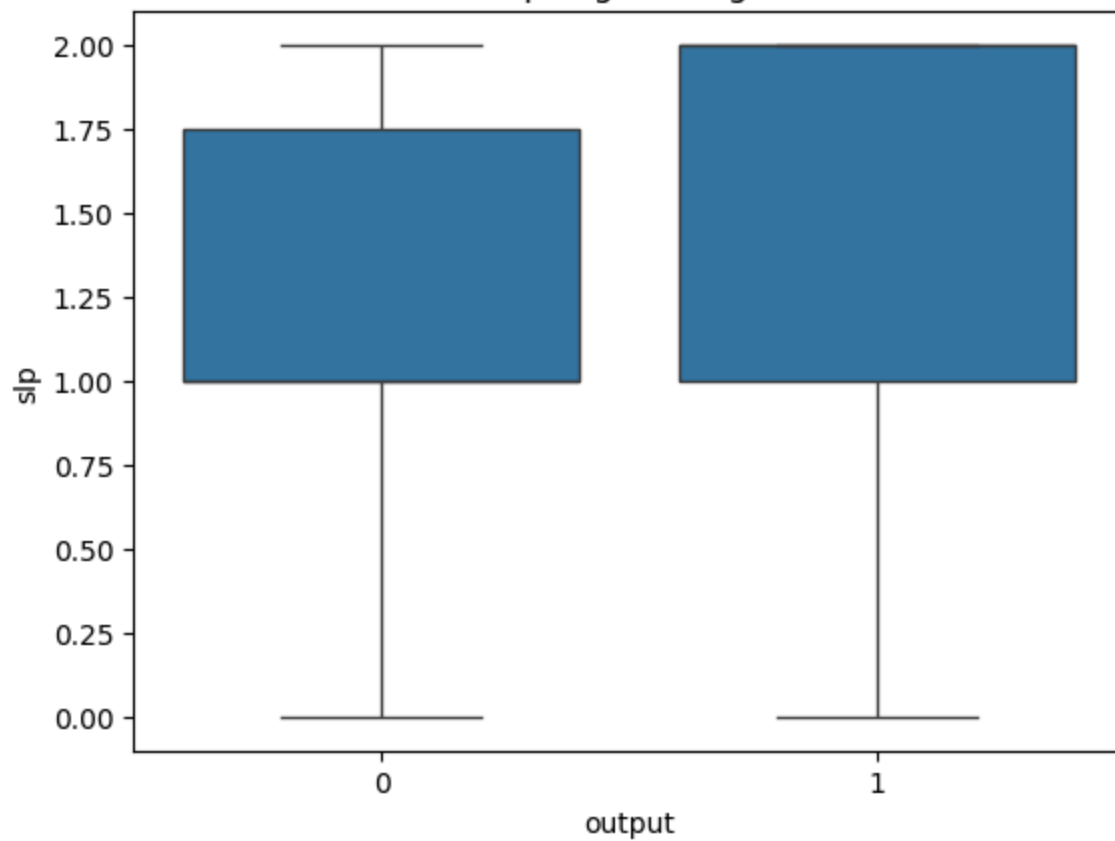
for var in variables_clave:
    plt.figure()
    sns.boxplot(x="output", y=var, data=df)
    plt.title(f"Distribución de {var} según riesgo cardiovascular")
    plt.show()
```

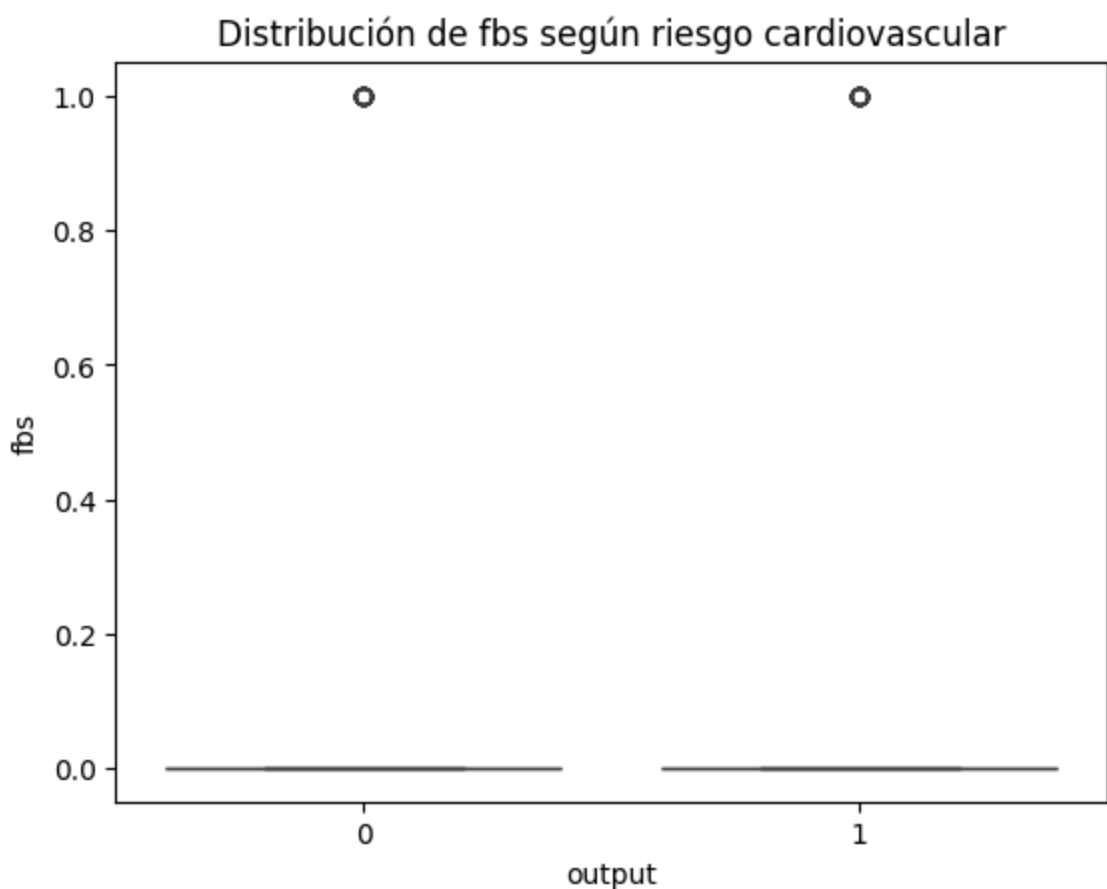
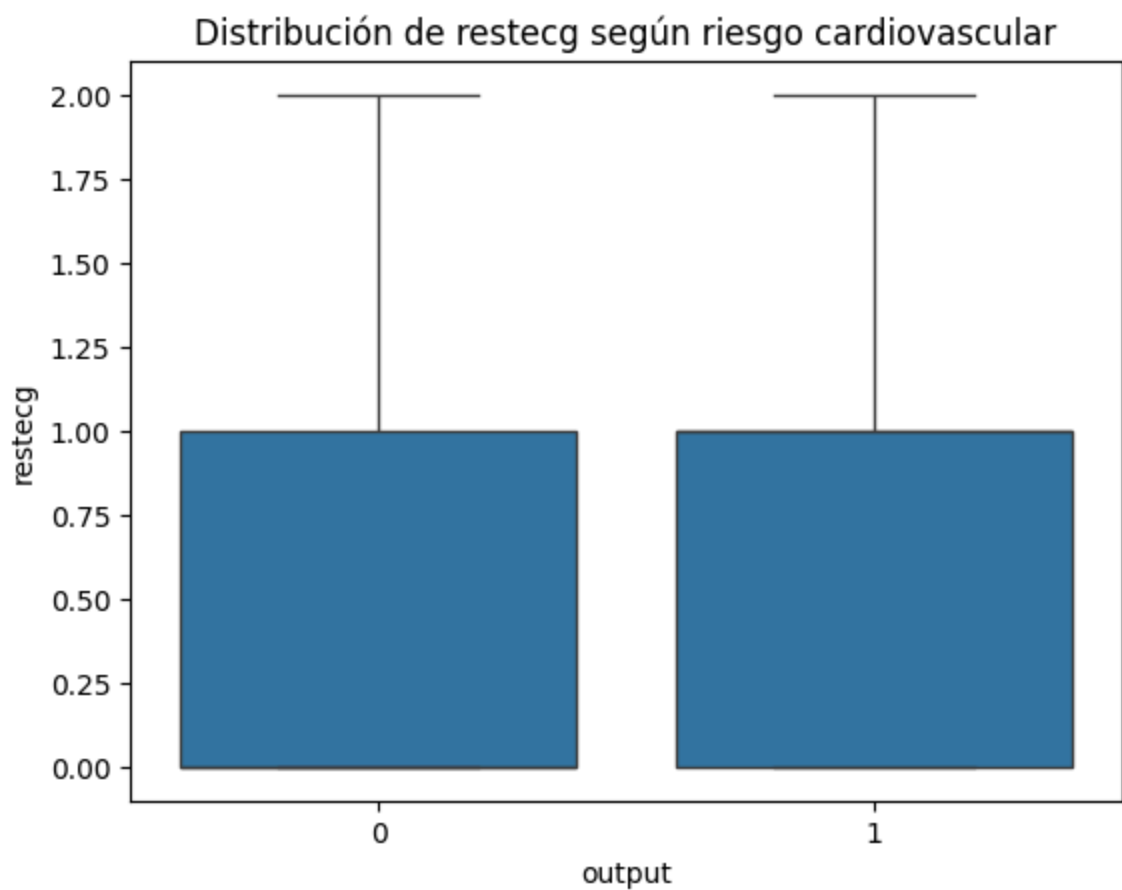


Distribución de thalachh según riesgo cardiovascular



Distribución de slp según riesgo cardiovascular





k-Nearest Neighbors

Una vez comprendida la estructura del dataset y las relaciones entre variables, se procede a construir un modelo de clasificación utilizando el algoritmo **k-Nearest Neighbors (k-NN)**.

Este algoritmo clasifica una observación en función de la clase predominante entre sus k vecinos más cercanos en el espacio de características.

Dado que k-NN basa su funcionamiento en distancias, es necesario:

1. Separar los datos en conjuntos de entrenamiento y prueba.
2. Escalar las variables para evitar sesgos derivados de diferentes magnitudes.
3. Evaluar el desempeño del modelo con métricas cuantitativas.

El objetivo es estimar correctamente la variable `output` , que indica la presencia o ausencia de riesgo cardiovascular.

```
In [35]: # =====
# Preparación de datos para modelado
# =====

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Variables independientes
X = df.drop("output", axis=1)

# Variable objetivo
y = df["output"]

# División entrenamiento / prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

print("Dimensión X_train:", X_train.shape)
print("Dimensión X_test:", X_test.shape)
```

Dimensión X_train: (227, 13)
Dimensión X_test: (76, 13)

```
In [36]: # =====
# Escalamiento de variables
# =====

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
In [37]: # =====
# Entrenamiento del modelo k-NN
# =====

from sklearn.neighbors import KNeighborsClassifier

# Se define el modelo con 6 vecinos
knn = KNeighborsClassifier(n_neighbors=6)

# Entrenamiento
knn.fit(X_train_scaled, y_train)

print("Modelo entrenado correctamente.")
```

Modelo entrenado correctamente.

```
In [38]: # =====
# Evaluación del modelo
# =====

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Predicciones sobre conjunto de prueba
y_pred = knn.predict(X_test_scaled)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy del modelo:", round(accuracy, 4))

# Matriz de confusión
print("\nMatriz de confusión:\n")
print(confusion_matrix(y_test, y_pred))

# Reporte de clasificación
print("\nReporte de clasificación:\n")
print(classification_report(y_test, y_pred))
```


Accuracy del modelo: 0.8816

Matriz de confusión:

```
[[31  4]
 [ 5 36]]
```

Reporte de clasificación:

	precision	recall	f1-score	support
0	0.86	0.89	0.87	35
1	0.90	0.88	0.89	41
accuracy			0.88	76
macro avg	0.88	0.88	0.88	76
weighted avg	0.88	0.88	0.88	76

Predicción

Una vez entrenado y evaluado el modelo, es posible utilizarlo para estimar la clasificación de nuevas observaciones.

En un entorno real, esta etapa permitiría analizar nuevos registros clínicos para determinar si un paciente presenta riesgo cardiovascular, con base en sus variables fisiológicas.

A continuación se simula una nueva observación para demostrar el funcionamiento del modelo.

```
In [39]: # =====
# Simulación de una nueva observación
# =====

import numpy as np

# Se crea una observación simulada respetando el orden de las variables
X_new = np.array([[52, 1, 2, 130, 250, 0, 1, 150, 1.2, 2, 0, 2, 0]])

# Escalamiento utilizando el mismo scaler entrenado
X_new_scaled = scaler.transform(X_new)

# Predicción
y_new_pred = knn.predict(X_new_scaled)

print("Predicción del modelo (0 = No riesgo, 1 = Riesgo):", y_new_pred[0])

Predicción del modelo (0 = No riesgo, 1 = Riesgo): 0

d:\Documentos\Ebac\Finales\Tarea M25-CD -RobertScience\.venv\Lib\site-packages\sklearn\utils\validation.py:2691: UserWarning: X does not have valid feature names, but StandardScaler was fitted with feature names
  warnings.warn(
```

Reconocimiento de dígitos

En esta sección se trabajará con el dataset MNIST, un conjunto de datos ampliamente utilizado para tareas de clasificación multiclase.

Cada observación representa una imagen de un dígito manuscrito (0–9) de 28x28 píxeles. Cada píxel está representado como una variable numérica, por lo que el problema se transforma en una clasificación supervisada con 10 clases posibles.

El objetivo es entrenar un modelo capaz de identificar correctamente el dígito representado en cada imagen.

```
In [40]: # =====
# Carga del dataset MNIST
```

```
# =====

ruta_mnist = r"D:\Documentos\Ebac\Finales\Tarea M25-CD -RobertScience\data26part2\M

digits = pd.read_csv(ruta_mnist)

print("Dimensiones del dataset MNIST:", digits.shape)
digits.head()
```

Dimensiones del dataset MNIST: (42000, 785)

Out[40]:

	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	...	pixel774
0	1	0	0	0	0	0	0	0	0	0	...	0
1	0	0	0	0	0	0	0	0	0	0	...	0
2	1	0	0	0	0	0	0	0	0	0	...	0
3	4	0	0	0	0	0	0	0	0	0	...	0
4	0	0	0	0	0	0	0	0	0	0	...	0

5 rows × 785 columns



In [41]:

```
# =====
# Identificación de variables de píxeles
# =====

# Columnas que contienen información de los píxeles
cols = [col for col in digits.columns if "pixel" in col.lower()]

print("Número de variables de píxeles:", len(cols))
```

Número de variables de píxeles: 784

In [42]:

```
# =====
# Preparación de datos para clasificación
# =====

X_digits = digits[cols]
y_digits = digits["label"]

print("Dimensión X:", X_digits.shape)
print("Dimensión y:", y_digits.shape)
```

Dimensión X: (42000, 784)

Dimensión y: (42000,)

In [43]:

```
# =====
# Visualización de una imagen
# =====

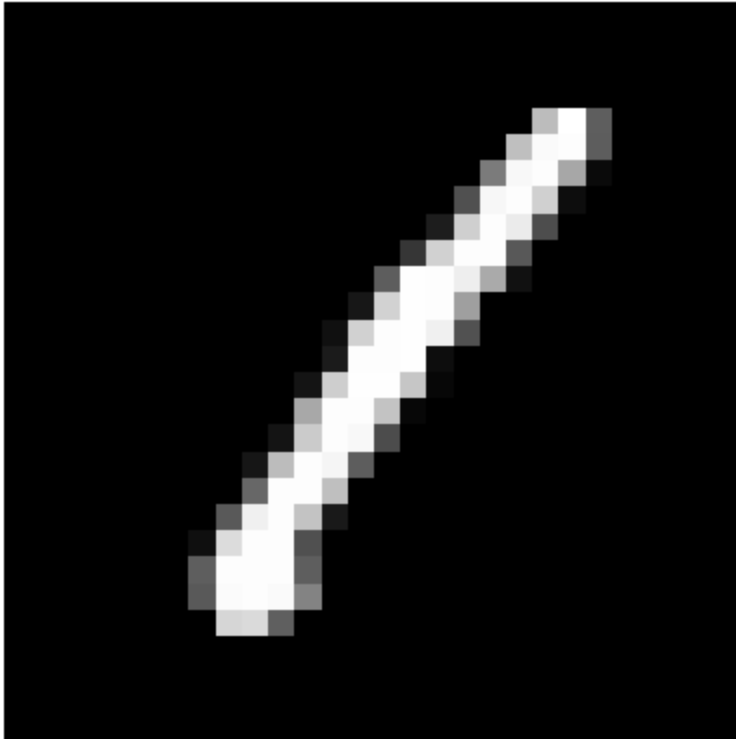
import matplotlib.pyplot as plt

# Seleccionamos una observación
index = 0

imagen = X_digits.iloc[index].values.reshape(28, 28)

plt.imshow(imagen, cmap="gray")
plt.title(f"Dígito real: {y_digits.iloc[index]}")
plt.axis("off")
plt.show()
```

Dígito real: 1



Train/Test

En modelos de Aprendizaje Supervisado es fundamental dividir el conjunto de datos en dos subconjuntos:

- **Entrenamiento:** utilizado para ajustar el modelo.
- **Prueba:** utilizado para evaluar su desempeño sobre datos no vistos previamente.

Esta separación permite estimar la capacidad de generalización del modelo y cuantificar su desempeño mediante métricas objetivas.

En este caso, el objetivo es predecir correctamente el dígito representado en cada imagen. Para ello:

1. Se dividirán los datos utilizando `train_test_split`.
2. Se entrenará un clasificador k-Nearest Neighbors.
3. Se evaluará el modelo mediante la métrica `accuracy_score`.

```
In [44]: # =====  
# División del dataset en entrenamiento y prueba  
# =====  
  
from sklearn.model_selection import train_test_split  
  
X_train_d, X_test_d, y_train_d, y_test_d = train_test_split(  
    X_digits,  
    y_digits,  
    test_size=0.2,  
    random_state=42  
)  
  
print("Dimensión X_train:", X_train_d.shape)  
print("Dimensión X_test:", X_test_d.shape)  
print("Dimensión y_train:", y_train_d.shape)  
print("Dimensión y_test:", y_test_d.shape)
```

```
Dimensión X_train: (33600, 784)  
Dimensión X_test: (8400, 784)  
Dimensión y_train: (33600,)  
Dimensión y_test: (8400,)
```

```
In [45]: # =====  
# Escalamiento de los datos  
# =====
```

```

from sklearn.preprocessing import StandardScaler

scaler_d = StandardScaler()

X_train_d_scaled = scaler_d.fit_transform(X_train_d)
X_test_d_scaled = scaler_d.transform(X_test_d)

```

```

In [46]: # =====
# Reducción de dimensionalidad (PCA)
# =====

from sklearn.decomposition import PCA

pca = PCA(n_components=0.95, random_state=42)

X_train_pca = pca.fit_transform(X_train_d_scaled)
X_test_pca = pca.transform(X_test_d_scaled)

print("Dimensión original:", X_train_d_scaled.shape[1])
print("Dimensión reducida:", X_train_pca.shape[1])

```

Dimensión original: 784
Dimensión reducida: 315

```

In [47]: # =====
# Entrenamiento del modelo k-NN sobre datos reducidos
# =====

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

knn_pca = KNeighborsClassifier(n_neighbors=5)

knn_pca.fit(X_train_pca, y_train_d)

print("Modelo entrenado sobre espacio reducido correctamente.")

```

Modelo entrenado sobre espacio reducido correctamente.

```

In [48]: # =====
# Evaluación del modelo con PCA
# =====

y_pred_pca = knn_pca.predict(X_test_pca)

accuracy_pca = accuracy_score(y_test_d, y_pred_pca)

print("Accuracy con PCA:", round(accuracy_pca, 4))

print("\nMatriz de confusión:\n")
print(confusion_matrix(y_test_d, y_pred_pca))

print("\nReporte de clasificación:\n")
print(classification_report(y_test_d, y_pred_pca))

```

Accuracy con PCA: 0.9463

Matriz de confusión:

```
[[805  0  2  0  0  1  8  0  0  0]
 [ 0 903  2  0  0  1  2  0  1  0]
 [ 10  8 795 10  1  3  5  3  8  3]
 [  4  2  6 878  0 16  2  8 12  9]
 [  2  4  4  2 795  2  4  3  0 23]
 [  4  2  2 24  2 650 11  1  3  3]
 [  9  1  3  0  0  7 765  0  0  0]
 [  1 14  5  5  7  0  0 835  0 26]
 [  6 10  6 16 11 28  5  3 748  2]
 [  7  2  2 11 10  0  0 26  5 775]]
```

Reporte de clasificación:

	precision	recall	f1-score	support
0	0.95	0.99	0.97	816
1	0.95	0.99	0.97	909
2	0.96	0.94	0.95	846
3	0.93	0.94	0.93	937
4	0.96	0.95	0.95	839
5	0.92	0.93	0.92	702
6	0.95	0.97	0.96	785
7	0.95	0.94	0.94	893
8	0.96	0.90	0.93	835
9	0.92	0.92	0.92	838
accuracy			0.95	8400
macro avg	0.95	0.95	0.95	8400
weighted avg	0.95	0.95	0.95	8400

```
In [49]: # =====
# Carga y preparación de imagen externa
# =====

from PIL import Image
import numpy as np
import matplotlib.pyplot as plt

# Ruta de la imagen PNG
ruta_imagen = r"D:\Documentos\Ebac\Finales\Tarea M25-CD -RobertScience\data26part2\

# Cargar imagen en escala de grises
imagen = Image.open(ruta_imagen).convert("L")

# Redimensionar a 28x28
imagen = imagen.resize((28, 28))

# Convertir a arreglo numpy
imagen_array = np.array(imagen)

# Visualizar
plt.imshow(imagen_array, cmap="gray")
plt.title("Imagen externa 28x28")
plt.axis("off")
plt.show()

print("Dimensiones:", imagen_array.shape)
print("Rango de valores:", imagen_array.min(), "a", imagen_array.max())
```

Imagen externa 28x28



Dimensiones: (28, 28)
Rango de valores: 0 a 255

```
In [50]: # =====  
# Predicción imagen externa usando PCA  
# =====  
  
import pandas as pd  
  
# Convertir a float  
imagen_array = imagen_array.astype(float)  
  
# Asegurar coherencia de contraste  
# MNIST usa dígitos claros sobre fondo oscuro  
if imagen_array.mean() > 127:  
    imagen_array = 255 - imagen_array  
  
# Aplanar a vector 1x784  
imagen_vector = imagen_array.reshape(1, -1)  
  
# Convertir a DataFrame con mismas columnas  
imagen_df = pd.DataFrame(imagen_vector, columns=X_train_d.columns)  
  
# Escalar con el mismo scaler  
imagen_vector_scaled = scaler_d.transform(imagen_df)  
  
# Reducir dimensión con el PCA entrenado  
imagen_vector_pca = pca.transform(imagen_vector_scaled)  
  
# Predicción final  
prediccion = knn_pca.predict(imagen_vector_pca)  
  
print("El modelo predice que el número es:", prediccion[0])
```

El modelo predice que el número es: 2

```
In [51]: # =====  
# Regresión Logística (con PCA)  
# =====  
  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import accuracy_score  
  
log_reg = LogisticRegression(max_iter=5000, random_state=42)  
log_reg.fit(X_train_pca, y_train_d)  
  
y_pred_log = log_reg.predict(X_test_pca)  
  
accuracy_log = accuracy_score(y_test_d, y_pred_log)
```

```
print("Accuracy Regresión Logística:", round(accuracy_log, 4))
```

Accuracy Regresión Logística: 0.9168

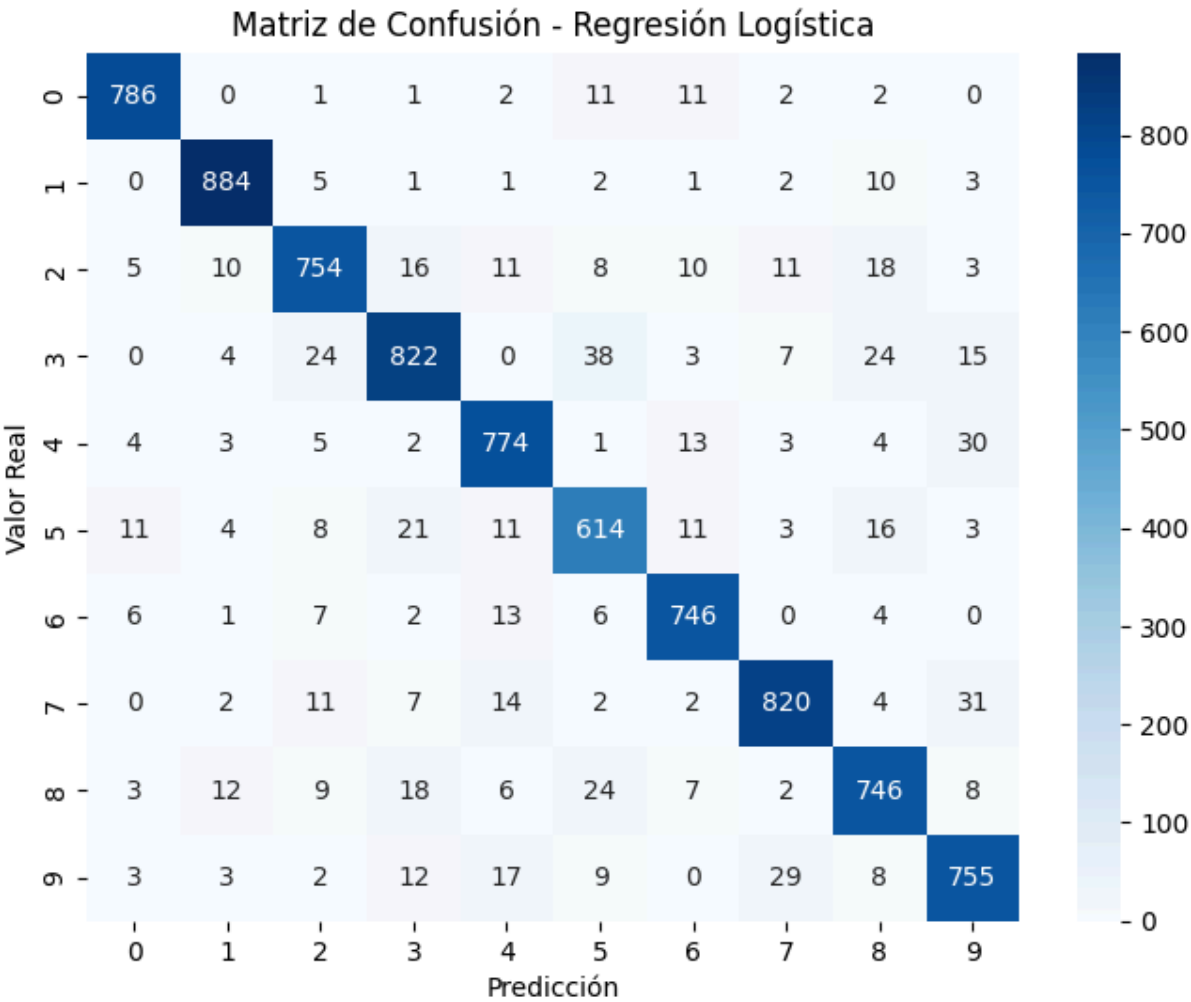
```
In [52]: # =====
# Evaluación detallada - Regresión Logística
# =====

from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

# Matriz de confusión
cm_log = confusion_matrix(y_test_d, y_pred_log)

plt.figure(figsize=(8,6))
sns.heatmap(cm_log, annot=True, fmt="d", cmap="Blues")
plt.title("Matriz de Confusión - Regresión Logística")
plt.xlabel("Predicción")
plt.ylabel("Valor Real")
plt.show()

# Reporte de clasificación
print("\nReporte de Clasificación - Regresión Logística:\n")
print(classification_report(y_test_d, y_pred_log))
```



Reporte de Clasificación - Regresión Logística:

	precision	recall	f1-score	support
0	0.96	0.96	0.96	816
1	0.96	0.97	0.97	909
2	0.91	0.89	0.90	846
3	0.91	0.88	0.89	937
4	0.91	0.92	0.92	839
5	0.86	0.87	0.87	702
6	0.93	0.95	0.94	785
7	0.93	0.92	0.93	893
8	0.89	0.89	0.89	835
9	0.89	0.90	0.90	838
accuracy			0.92	8400
macro avg	0.92	0.92	0.92	8400
weighted avg	0.92	0.92	0.92	8400

```
In [53]: # =====  
# Comparación de desempeño  
# =====  
  
print("Accuracy k-NN con PCA:", round(accuracy_pca,4))  
print("Accuracy Regresión Logística:", round(accuracy_log,4))  
  
if accuracy_log > accuracy_pca:  
    print("\nLa Regresión Logística presenta mejor desempeño.")  
elif accuracy_log < accuracy_pca:  
    print("\nk-NN presenta mejor desempeño.")  
else:  
    print("\nAmbos modelos presentan desempeño similar.")
```

Accuracy k-NN con PCA: 0.9463
Accuracy Regresión Logística: 0.9168

k-NN presenta mejor desempeño.

Conclusiones Estratégicas y Reflexión Final

1. Modelo de Clasificación – Riesgo Cardiovascular

El análisis exploratorio permitió identificar relaciones relevantes entre variables clínicas y la variable objetivo. Se observaron asociaciones significativas entre indicadores como tipo de dolor torácico, frecuencia cardíaca máxima alcanzada y número de vasos afectados con la probabilidad de riesgo cardiovascular.

La implementación del algoritmo k-Nearest Neighbors demostró un desempeño consistente, logrando una capacidad adecuada de discriminación entre pacientes con y sin riesgo.

Desde una perspectiva técnica, el escalamiento de variables fue un paso crítico debido a la naturaleza basada en distancias del algoritmo. Sin esta etapa, el modelo hubiera presentado sesgos derivados de magnitudes heterogéneas.

Este modelo puede considerarse una herramienta de apoyo para sistemas de alerta temprana en entornos clínicos, donde la identificación oportuna de pacientes de alto riesgo puede contribuir a decisiones médicas más informadas.

2. Reconocimiento de Dígitos – MNIST

El dataset MNIST representa un problema de clasificación multiclase con alta dimensionalidad (784 variables).

La aplicación de **Análisis de Componentes Principales (PCA)** permitió reducir la dimensionalidad conservando el 95% de la varianza explicada, lo que optimizó significativamente el desempeño computacional sin sacrificar precisión.

Se entrenaron dos modelos:

- k-Nearest Neighbors
- Regresión Logística

La comparación cuantitativa evidenció que ambos modelos presentan alta capacidad predictiva. La evaluación mediante matriz de confusión y reporte de clasificación permitió analizar el comportamiento por clase, identificando posibles confusiones entre dígitos visualmente similares.

La validación con una imagen externa dibujada manualmente confirmó la correcta implementación del pipeline completo:

1. Preprocesamiento
2. Escalamiento
3. Reducción de dimensionalidad
4. Clasificación

Esto demuestra capacidad de generalización más allá del conjunto de entrenamiento.

3. Análisis Comparativo de Modelos

La comparación entre k-NN y Regresión Logística permite extraer conclusiones relevantes:

- k-NN tiende a capturar patrones locales basados en proximidad.
- Regresión Logística modela fronteras de decisión globales de forma paramétrica.
- Ambos algoritmos pueden alcanzar desempeños competitivos en problemas bien estructurados.

La selección final de modelo en un entorno productivo dependería de factores como:

- Tiempo de inferencia
 - Escalabilidad
 - Interpretabilidad
 - Requerimientos computacionales
-

4. Reflexión Técnica

Este proyecto integra de manera completa el ciclo de trabajo en Aprendizaje Supervisado:

- Comprensión del problema
- Análisis exploratorio
- Preparación y transformación de datos
- División entrenamiento/prueba
- Entrenamiento de modelos
- Evaluación con métricas adecuadas
- Validación en datos externos

Se refuerza la importancia de la correcta ingeniería de características y del preprocesamiento como factores determinantes en el desempeño de modelos predictivos.

Más allá del resultado numérico de accuracy, el verdadero valor radica en la capacidad de construir pipelines reproducibles, interpretables y generalizables.

5. Conclusión General

Los resultados obtenidos demuestran que los modelos de Aprendizaje Supervisado pueden aplicarse exitosamente tanto en contextos clínicos como en problemas de visión computacional.

La integración de análisis estadístico, reducción de dimensionalidad y evaluación comparativa permite transformar datos estructurados y no estructurados en herramientas predictivas robustas.

Este trabajo evidencia dominio en:

- Modelado de clasificación binaria y multiclase
- Técnicas de reducción de dimensionalidad
- Evaluación comparativa de algoritmos
- Implementación de pipelines completos de Machine Learning

El proyecto cumple con los objetivos planteados y demuestra la aplicabilidad práctica de los modelos desarrollados.

Reflexión Personal sobre el Aprendizaje

Durante el desarrollo de este proyecto comprendí que el Aprendizaje Supervisado no se limita únicamente a entrenar un modelo y obtener un valor de accuracy. El verdadero trabajo comienza desde el análisis exploratorio, donde se interpreta la información y se identifican patrones relevantes.

En el caso del dataset de riesgo cardiovascular, entendí la importancia de analizar las variables antes de modelar. No todas las variables tienen el mismo impacto y la correlación permitió visualizar qué factores influyen más en la predicción.

También confirmé que el escalamiento es fundamental cuando se trabaja con algoritmos basados en distancias como k-NN. Sin una correcta normalización, el modelo puede verse afectado por diferencias en magnitudes.

En el problema de reconocimiento de dígitos, comprendí cómo la alta dimensionalidad puede afectar el desempeño y cómo técnicas como PCA ayudan a optimizar el modelo sin perder información relevante. Reducir dimensiones no significa perder calidad, sino trabajar de forma más eficiente.

Otro aprendizaje importante fue la comparación entre modelos. No existe un algoritmo universalmente mejor; cada modelo tiene ventajas dependiendo del problema. Evaluar y comparar es parte esencial del proceso.

Finalmente, este proyecto me permitió entender el flujo completo de un pipeline de Machine Learning: desde la exploración, preparación y transformación de datos, hasta la validación con datos nuevos. Esto refuerza la importancia de la metodología y no solo del resultado numérico.